Six Pitfalls in Measuring the Cost of Coding Agents: Evidence from a Cross-provider Study of Code Mode

Max Reischl *reischl_max@icloud.com*
Independent researcher, Vienna, Austria

Alexander Feurer *feureralexander10@gmail.com*
Independent researcher, Vienna, Austria

Abstract

Agent benchmarks now report dollars beside accuracy, but cost is usually estimated from token counts and a single provider's price list, on one task suite, in one run, in one execution order. We ran a study of code mode—a harness's programmatic tool-calling mode against its native tool calling—across six models from five providers, alongside four other open-source harnesses, on two task suites (8,520 planned cells). The protocol was fixed in advance and amended 23 times (Appendix A); a replication criterion and a counterbalanced re-run plan were filed before those data existed. We metered every request at a gateway that records each provider's usage and cache accounting, validated to 0.023% on one 62-run Anthropic development pilot whose raw logs are not in the checksummed artifact. On the second suite, code mode (PTC against native) was cheaper for DeepSeek and gpt-6-luna: −21.8% [−37.46, −9.38] and −16.6% [−24.35, −8.55] per task (both intervals exclude zero and both survive Holm), and −22.64% [−39.77, −9.19] and −18.01% [−25.14, −10.61] on cells both modes solved. A counterbalanced re-run reproduced the per-task savings (−16.8% [−29.8, −6.3] and −19.0% [−29.0, −9.4]). Neither saving appears on batch 1. We use the study to quantify six pitfalls. (1) *Token volume is not cost*: pricing tokens without cache accounting reverses the point-estimate sign of the code-mode effect for three of six models, whose billed intervals include zero; after Holm correction only Haiku survives on batch 1. (2) *Cache configuration is part of the treatment*: where the cache breakpoint sat decided whether code mode was cached at all, and moving it made the same matrix 3.4 times cheaper. (3) *Accounting can be missing*: on one provider's compatibility endpoint 128.1M input tokens arrived without a cache split, and the study's own cost tool reported more money than was spent. (4) *Execution order carries cost*: one configuration's cost varied 1.75–3.82-fold with its slot; the order effect was significant in 1 of 9 re-run rows, and a counterbalanced re-run did not bring the reversed GLM saving back (within the −8.1% to +8.0% native-arm drift). (5) *Cost without quality misleads*: on one model, code mode looked 54% dearer per task but was 101% dearer per solved task, because it also solved fewer tasks (46 against 60 of 62), and the re-run reproduced the gap (48 against 62). (6) *One suite is not enough*: two interleaved collections of identical requests differed by −5.3%, with an interval half-width of 11.6% at 31 tasks, and our own pre-registered replication on a second suite failed for all four providers. On Claude Haiku 4.5, code mode is dearer and less reliable on batch 1 (the same per-solved direction appears on the second suite, but those intervals include zero). We give a checklist for cost measurement and release the gateway, logs and analysis code.

1 Introduction

Cost has become a first-class outcome in agent evaluation. Accuracy-only leaderboards reward needlessly expensive agents (Kapoor et al., 2024); cost-of-pass combines price and success (Erol et al., 2025); leaderboards now log per-task cost across tens of thousands of rollouts (Kapoor et al., 2025); and coding-agent papers

report dollar cost on SWE-bench (Jimenez et al., 2024; Xia et al., 2025; Zhang et al., 2024) (about $0.43 per Lite task for AutoCodeRover). Agentic coding consumes orders of magnitude more tokens than code chat, and runs of the same task vary widely (Bai et al., 2026), so the stakes are real.

How cost is *measured*, however, has received far less scrutiny than how accuracy is measured. A typical cost figure multiplies token counts by one provider's list prices, sometimes assuming no prompt caching (Xu et al., 2024), on one task suite, from one run per configuration, with configurations run in a fixed order. Each of these choices can bias a cost comparison, and none is usually tested.

We tested them in the course of a study of *code mode*, a design in which an agent writes a program that calls its tools instead of issuing one tool call per model request (Wang et al., 2024). Vendors report large token savings from code mode (Anthropic, 2025b;a; Cloudflare, 2026), while academic work finds the saving depends on workload (Patel et al., 2026). We compared the programmatic tool-calling mode of an open-source harness with its native mode, on six models from five providers, alongside four other harnesses, on two task suites. A gateway between every harness and every provider recorded each provider's own usage block, including the cache split, for every request.

The study's answer about code mode is mixed. On batch 1 no model was significantly cheaper under code mode; on the second suite DeepSeek and gpt-6-luna showed a saving that a counterbalanced re-run reproduced (§9). The filed replication of the batch-1 pattern failed. The more general result is what it took to reach that answer. At six points a standard measurement practice would have changed the conclusion or mis-sized it, and we can quantify each one with the study's own data.

Contributions.

1. Six pitfalls in measuring agent cost, each quantified on a real study (§5–§8): three in cost accounting, one in experimental design, one in the outcome measure, and one in inference.
2. A checklist that addresses each one (§10).
3. A gateway-metered, cross-provider measurement of code mode's billed cost, validated against provider billing, with a pre-registered replication reported although it failed, and a pre-registered re-run of the central contrast under counterbalanced order (§6.1, §9).
4. The gateway, run records, logs and analysis code, with every reported number re-derivable (Appendix F). Unlike the largest public agentic trace set, which records anonymised providers and cached-token counts from human-driven sessions (Nixon et al., 2026), our logs come from controlled matrices, name each provider, keep the cache-write count where the provider reports one, and were checked against the provider's bill where its balance could be read.

2 Related work

Cost in agent evaluation. Beyond the work above, Efficient Agents decomposes framework design into cost-of-pass (Wang et al., 2025). The closest work, a preregistered benchmark, treats prompt wording, inference effort and harness policy as interacting factors and measures cost per successful task, including runs on DSH (Weinberger & Hozez, 2026). We share its outcome measure but study the measurement itself: billed against priced cost, execution order, a noise floor and replication. An enterprise router study argues that a harness's defaults, caching among them, set the bill (Kwartler et al., 2026). Production gateway data report per-agent cache hit rates across coding agents (Jaigu, 2026), and an independent CLI benchmark finds that several agents do not cache at all (Kalelioğlu & Dilmegani, 2026). A cheaper list price does not guarantee a cheaper run (Chen et al., 2026), and routers exploit price differences between models (Ong et al., 2025; Chen et al., 2024; Ding et al., 2024; Yang et al., 2026b).

Validity of agent evaluation. Harness choice alone can change tokens per solved task 40-fold (Vats & Golev, 2026), and changing only a harness's context handling, with model and task fixed, changes solve rates (Lewis, 2026); harness gains are comparable to model gains and harness effects are heterogeneous across models (Han & Sun, 2026); harness-induced variance can exceed model variance (Zhang et al., 2026b); a controlled

pairing study finds resolve rates moving by up to 44 points and charged tokens per resolved task by more than 20-fold across harnesses for the same model (Zhao et al., 2026); a factorial harness-design study varies planning, action space and context management with cost effects (Fan et al., 2026); and leading SWE-bench entries are statistically inseparable (Liu et al., 2026). One study finds rankings stable under scaffold shift (Ndzomga, 2026). Nominally deterministic agent runs vary by several points of pass@1 (Bjarnason et al., 2026), and run-count guidance exists for accuracy but not for cost (Blackwell et al., 2024; Potamitis et al., 2025); a survey finds that no prominent agent benchmark reports a complete standardised cost record (Nageshwaran et al., 2026). Repeated configurations vary in per-cell cost with coefficients of variation of 2.4–8.4% (Hu et al., 2026), and A/A designs have been proposed for accuracy (Kaliyev & Maryanskyy, 2026). Failed replications of LLM evaluation claims are published (Schaeffer et al., 2023; Mirzadeh et al., 2025; Xu & Wu, 2026).

Caching and its measurement. Claw-SWE-Bench v1 reports per-harness billed cost from provider billing logs together with per-harness cache hit rates (Zheng et al., 2026a), and a gateway benchmark publishes per-arm hit rates from spend logs (LiteLLM team, 2026). Caching cuts agent cost by 41–80% across sessions (Lumer et al., 2026); identical runs on open-weight models show harness-dependent cache regimes (no de la Rubia & Pafka, 2026); and a preregistered benchmark reports that hits are probabilistic and that one provider route returned no cached tokens for days (Weinberger & Hozez, 2026). An empirical study of 120 prompt-caching pull requests finds that mis-integrations are common, often silent, and provider-API heterogeneous (Zhang et al., 2026a). Anthropic documents that its OpenAI-compatible layer does not support prompt caching (Anthropic, 2025c), and issue reports observed the effect in harnesses (cline, 2026; NousResearch, 2026); other routers forward cache markers (OpenRouter, 2026). Code minification can cut state-in-context agent input tokens substantially on SWE-bench Verified, with a corresponding drop in resolve rate (Hrubec & Cito, 2026). Serving-side prompt reuse is well studied (Gim et al., 2024; Yao et al., 2025; Jiang et al., 2024).

Code mode and tool interfaces. Executable code as an action space was introduced by CodeAct (Wang et al., 2024). A synthetic study finds a token crossover near a fan-out of 26 calls (Patel et al., 2026); a five-interface study places programmatic calling between typed calls and bash (Mak et al., 2026); restricting coding agents to one code-execution tool holds pass rates while costs swing 20–40% (Yang et al., 2026a). Vendor demonstrations report order-of-magnitude token savings when the alternative is stuffing large MCP catalogs into context (150,000→2,000 tokens, 98.7%; or ∼99.9% fewer input tokens for a 2,500-endpoint API) (Anthropic, 2025b; Cloudflare, 2026); those figures measure progressive disclosure of huge HTTP APIs, not billed cost of programmatic versus native calling inside a coding harness with a small tool catalog. The closest published head-to-head, on six short deterministic workflows and eight models, finds Code Mode cut mean tokens by 37.6% and mean cost by 40.7% versus native MCP, with regressions on weaker and Gemini-family models (Mbat et al., 2026); that average saving is larger than ours, but the capability-dependent sign matches, and their workloads are not multi-file coding tasks metered at a multi-provider gateway. E-Bench reports tool-call and turn cuts across eleven models on synthetic product tasks (MCP calls −73.8%, turns −33.6%), not cache-aware cross-provider billed dollars (Zheng et al., 2026b); a vendor CodeAct agent likewise reports fewer input and output tokens on an in-house benchmark without a published cache-aware bill reconciliation (Strands Agents team, 2026). Large-scale tool-use work studies retrieval over many APIs (Qin et al., 2024), and a Seal-Tools baseline shows retrieval accuracy falling as the tool set grows (Lumer et al., 2025); interface design affects success (Yang et al., 2024; SWE-agent team, 2025). Concurrent work reprices three harnesses under two vendors' price lists (Liu & Han, 2026).

What is new here is not that cost can be measured at a gateway or that caching matters, both of which are established, but a quantification of how standard measurement choices change a cost conclusion, on one cross-provider study with a living, amended protocol.

3 The study

3.1 Harnesses and arms

An *arm* is a harness in a fixed configuration; ten arms ran on every model. Six are conditions of DeepSeek Harness (DSH, version 0.1.7-rc.2), DeepSeek's open-source coding harness. In its *native* mode the model sees

24 tool schemas and calls one tool per request. In its *PTC* (programmatic tool calling) mode the tools are presented as a generated TypeScript SDK behind a single `run_code` tool, and the model writes a program that calls them. The archived requests confirm 24 and 1 schemas. The six DSH arms cross native/PTC with a context policy (shipped, or optimized with a tool-result pruner) and a terseness instruction. The central contrast is `dsh:ptc` against `dsh:native`. The other four arms are OpenCode 1.18.30, Cline 3.0.65, OpenHands 1.16.0 and Goose 1.52.0, run with their own defaults. No harness was patched. We did not develop DSH and are not affiliated with DeepSeek, but we wrote the DSH study conditions and use DSH daily, an asymmetry against the external harnesses that we cannot remove.

3.2 Models, providers and cache regimes

Six models from five providers (Table 1) each form one *matrix* of 10 arms × tasks × 2 seeds. Four providers contribute one model each, so model and provider are confounded except within Anthropic. The cache regime differs by provider: DeepSeek and Z.ai cache automatically; Qwen, served through Alibaba Cloud's Anthropic-compatible endpoint, receives a gateway-injected breakpoint on the last tool; on gpt-6-luna the requests travel the OpenAI Responses path, where configured breakpoints are inert; and on Anthropic the gateway injects static breakpoints plus the top-level cache field (§5.2). Reasoning effort was forced to high for DeepSeek and GLM and left unset for the others.

Table 1: Models, price cards (USD per million tokens) and cache regimes.

model	provider	miss	hit	output	out:in	discount	cache regime
DeepSeek V4.1 Flash	DeepSeek	0.15	0.003	0.60	4.00	50×	automatic
GLM-5.3-Flash	Z.ai	0.15	0.03	0.50	3.33	5×	automatic
Qwen3.8-Flash	Alibaba Cloud	0.15	0.016	0.47	3.13	9.4×	breakpoint on last tool
gpt-6-luna	OpenAI	0.10	0.01	0.50	5.00	10×	automatic
Claude Haiku 4.5	Anthropic	1.00	0.10	5.00	5.00	10×	static + top-level
Claude Sonnet 5.5	Anthropic	2.00	0.20	10.00	5.00	10×	static + top-level

3.3 Task suites and population

Batch 1 has 31 tasks in five strata (single-step, multi-file, repo-wide, long-horizon, fetch-heavy) plus a smoke task. Each task is a repository with a prompt and a test suite that fails on the untouched fixture and is deterministic. A run is *solved* if the harness exits 0 and the suite passes. The suite runs in the agent's workdir, so an agent can weaken the tests it is graded by (§7.1). *Tasks-b* has 40 tasks, ten in each of four strata. It was written after the batch-1 results were known, as a replication suite, and is graded by a hidden suite that the agent cannot read or edit.

Batch 1 has 3,720 planned cells (6 models × 31 tasks × 10 arms × 2 seeds). The Qwen log contains 621 records because one cell (`multi-file-feature-flags`, `dsh:native`, seed 2) was written twice (an infrastructure failure plus its retry); that failed record is one of two pre-registered exclusions. The other exclusion drops one luna cell. After both exclusions there are 3,719 analysed cells (Qwen 620, luna 619, the other four models 620). Tasks-b has 4,800 planned cells. Luna lost one cell, and Haiku lost 447 when the Anthropic account reached its monthly limit mid-matrix, so Haiku's tasks-b row uses the 353 analysed cells from the 18 tasks that ran first (7 of the 360 cells those tasks would have produced are missing).

3.4 Measurement

Each harness talks to the provider through an HTTP gateway and holds only a synthetic credential that encodes the run identifier, so every request's usage joins to its run by construction. The gateway substitutes the real key, enforces the model, logs every rewrite it applies, and records the provider's usage block. Per run,

$$\text{cost_billed} = \text{fresh} \cdot p_{\text{miss}} + \text{created} \cdot p_{\text{write}} + \text{hit} \cdot p_{\text{hit}} + \text{output} \cdot p_{\text{out}},$$

re-derived from the archived token counts. *cost_cold* prices every input token at the miss rate. Runs execute in a fresh fixture copy, four at a time, with a 20-minute timeout.

Validation against the bill. On one 62-run Anthropic development pilot we read the account's credit balance before and after and computed the pilot's cost from its own gateway log: $35.5718 computed against a $35.58 credit delta, a ratio of 1.00023 (0.023% relative). The pilot's raw logs are not in the checksummed artifact (Appendix F). For the main Anthropic matrices the same ledger bounds the cost from outside (§5.3). The re-run's Haiku matrix, on DSH only, agreed with the balance to the cent (Appendix D).

Design and analysis. Within each (task, seed) block the ten arms run in an order fixed by a seeded shuffle (§6.1). Hypotheses were filed before the six-model matrices ran (Appendix A), and a replication criterion for tasks-b was filed before any tasks-b data existed. The unit is the run, and tasks block. Contrasts are paired by task, with a percentile bootstrap over tasks ($B = 200{,}000$) on the absolute difference, expressed as a percentage of the native mean. A contrast is *positive* or *negative* if its 95% interval excludes zero, and *null* otherwise. Tests use sign-flip permutation (cost) or exact McNemar (solve rate), with Holm correction across the filed family.

4 A first reading

Table 2 is the result a standard analysis would report: the paired, raw, per-task cost of PTC against native on batch 1. Read at face value, it says that code mode is significantly dearer on two models (Qwen +30.7%, Haiku +53.8%), significantly cheaper on one (GLM −20.8%), and neutral on three. It also suggests a story: the three cases where code mode costs most or least differ in the provider's output:input price ratio. The rest of this paper shows how each of six measurement choices bears on that reading.

Table 2: The first reading: PTC against native on batch 1, raw paired per-task cost, percent of the native mean, with 95% task-clustered bootstrap intervals.

model	out:in ratio	PTC vs native	95% CI
GLM-5.3-Flash	3.33	−20.75%	[−44.30, −2.79]
DeepSeek V4.1 Flash	4.00	+0.13%	[−8.90, +9.76]
gpt-6-luna	5.00	+1.20%	[−8.42, +11.29]
Claude Sonnet 5.5	5.00	+9.51%	[−0.08, +18.01]
Qwen3.8-Flash	3.13	+30.67%	[+10.02, +53.79]
Claude Haiku 4.5	5.00	+53.84%	[+31.25, +76.73]

5 Pitfalls in cost accounting

5.1 Pitfall 1: token volume is not cost

Pitfall 1. A token count priced at list rates, without the provider's cache accounting, can give the wrong sign for a cost difference.

Providers discount cached input by factors from 5 to 50 (Table 1), and agent loops re-send most of their input. Table 3 compares the billed effect of PTC with its cold-cache counterpart, which prices every input token at the miss rate, as a token-count analysis without caching would. For three of six models (DeepSeek, gpt-6-luna, Sonnet) the point estimates have opposite signs. A token-volume analysis would report code mode as 10–21% cheaper on these models; their billed estimates are null (intervals include zero). After Holm correction of the billed family, only Haiku survives on batch 1. On Haiku both readings agree on the sign but differ by a factor of nearly three (+18.7% against +53.8%).

Table 3: PTC against native on batch 1, billed (with the provider's cache accounting) and cold (every input token at the miss rate). Point estimates, percent of native mean.

model	billed	cold	same sign?
Qwen3.8-Flash	+30.67%	+20.98%	yes
GLM-5.3-Flash	−20.75%	−20.55%	yes
gpt-6-luna	+1.20%	−21.17%	no
DeepSeek V4.1 Flash	+0.13%	−9.76%	no
Claude Haiku 4.5	+53.84%	+18.67%	yes
Claude Sonnet 5.5	+9.51%	−15.11%	no

The reason is that code mode saves input volume, and on providers with deep cache discounts input volume is cheap, so the saving is small in dollars while any extra output is paid in full. Token volume and billed cost answer different questions, and both should be reported.

5.2 Pitfall 2: cache configuration is part of the treatment

Pitfall 2. Where a provider requires the client to declare cache boundaries, the declaration decides what gets cached, and it can favour one arm over another.

Anthropic caches only prefixes the client marks, above a minimum length. Our gateway first marked the end of the tool list. PTC presents one tool, so its marked prefix fell under Haiku's 4,096-token minimum and was never cached, while native's 24-tool prefix was. Table 4 shows a pilot A/B on the same 31 tasks and two arms that changed only the cache declaration. With static breakpoints PTC's hit rate was 0.0% against native's 27.6%, and PTC cost 92% more. With the top-level cache field both arms cached 99.9%, PTC cost 80% more, and the 62 runs became 3.4 times cheaper. Under static breakpoints PTC cost $23.00 against native's $11.96. Moving the breakpoint cut the 62-run total from $34.96 to $10.42 (3.4×) and left PTC +79.5% dearer; PTC alone fell from $23.00 to $6.69.

Table 4: Pilot A/B on Claude Haiku 4.5 (31 tasks, `dsh:native` and `dsh:ptc`, one seed), changing only the cache declaration. From the development pilots; not part of the checksummed artifact.

	static breakpoints	+ top-level cache field
PTC cache hit rate	0.0%	99.9%
native cache hit rate	27.6%	99.9%
PTC cost per run	$0.74186	$0.21593
native cost per run	$0.38580	$0.12026
PTC against native	+92.3%	+79.5%
62-run total	$34.96	$10.42

The main matrices use the second condition. The same issue appears elsewhere in the design. On Qwen the gateway's breakpoint sits on the last tool, the very block PTC redefines, so cache loss under PTC there (retention 0.674 against 0.743) cannot be separated from breakpoint placement (§9). On luna the configured breakpoints are inert because the Responses API path ignores them. And the injected condition cannot be used with a client that declares its own breakpoints. In a later pilot with OpenCode on Anthropic's native API, the combined declarations exceeded Anthropic's limit of four and every request was refused. What looks like one treatment across providers is in fact four cache regimes, and each regime belongs to the experimental condition and should be reported with it.

5.3 Pitfall 3: accounting can be missing, and meters fail silently

Pitfall 3. A usage record without a cache split cannot be priced, and a cost tool that silently fills the gap with a default reports a number that is not a measurement.

The four external harnesses reach Anthropic through its OpenAI-compatible endpoint, which does not report a cache split. In the two batch-1 Anthropic matrices, 6,669 requests carrying 128.1M input tokens have no recorded cache state: a third of Haiku's input and half of Sonnet's. Two of the study's own tools filled the gap in opposite directions. One priced the tokens at zero. The shipped cost panel priced them at the miss rate and reported $266.87 for the pair, *more than the $144.33 the account was actually charged* for both matrices plus a pilot ($143.35 for Haiku alone). Only the outside ledger exposed this. Reconciled against it, the pair cost between $114.80 and $144.33, and the ledger implies that only about 19% of the unrecorded input was read from cache, against 93% for the DSH arms. Table 5 reports per-model brackets. We quote no standalone external-harness cost on Anthropic. The rankings and Figure 3 still include those arms, priced at the miss-rate default for unsplit input, and should be read with that caveat. A ten-cell pilot that ran OpenCode on Anthropic's native API, where the split is reported, measured a prefix retention of 0.874 [0.818, 0.930], below the DSH arms' 0.994–1.000.

Table 5: Cost per run and solve rate on batch 1 (all ten arms). For the four cheaper providers the USD totals and per-run figures are gateway-log totals, a slight upper bound because they include preflight requests, and are not recomputed from analysed-cell sums. Solve counts use analysed cells (luna 619, Qwen 620, the others 620). For Anthropic it is a bracket: each model's lower bound, and as upper bound the ledger minus the other model's lower bound.

model	total (USD)	per run (USD)	solved
gpt-6-luna	3.89	0.0063	589/619 (95.2%)
GLM-5.3-Flash	4.88	0.0079	617/620 (99.5%)
DeepSeek V4.1 Flash	4.95	0.0080	619/620 (99.8%)
Qwen3.8-Flash	12.44	0.0201	605/620 (97.6%)
Claude Sonnet 5.5	50.03–79.56	0.0807–0.1283	618/620 (99.7%)
Claude Haiku 4.5	64.77–94.30	0.1045–0.1521	519/620 (83.7%)

The study's development record contains three more silent failures of the same kind, all caught before the main matrices and fixed with tests. These come from the development record rather than the artifact. First, the gateway wrote Anthropic model names with hyphens where the price table used dots. The lookup fell back to the DeepSeek price card, and the pipeline reported $5.1523 for a pilot that cost $35.5718, a 6.90-fold undercount; at that factor the pipeline's 90% abort ($32.40) would have allowed about $220 of real spend. Second, the DeepSeek price card, copied from a third-party blog, was wrong in every cell. Correcting it moved DeepSeek's output:input ratio from 2.00 to 4.00 and its cache discount from 5× to 50×, which reversed the direction of the study's primary hypothesis on the data then available. Third, a regular expression that split run identifiers misattributed the usage of 632 records in one session, which then measured zero cost and looked plausible. The common feature is that each default produced a number, not an error.

6 Pitfall in experimental design

6.1 Pitfall 4: execution order carries cost

Pitfall 4. When configurations run in a fixed order, the order can carry a cost effect as large as the treatment, and a shared order turns it into bias.

Within each (task, seed) block the ten arms ran in an order set by a seeded shuffle. The seed depended on task and seed only, so *every matrix, including the replication suite, used the same permutation.* The assignment is balanced across blocks, but it is not randomised independently per matrix, and it is not balanced for any one contrast: `dsh:ptc` occupied the first slot in 2 of 62 blocks. Execution order turned out to matter. Within a single arm, mean cost varied by a factor of 1.75 to 3.82 across the ten slots, and the first slot was the cheapest in five of six models. We do not know why. Provider-side caching shared between arms, rate limiting and drift over the collection window are all candidates. A warming account alone does not predict a cheap first slot.

To estimate the contrast net of slot, we model the within-block difference as $d = c_{\text{ptc}} - c_{\text{native}} = \alpha + f(p_{\text{ptc}}) - f(p_{\text{native}})$, with f a step function over slots, on the log and the additive scale (task-clustered bootstrap,

$B = 20{,}000$). Table 6 sets the result beside the first reading. Two classifications move. GLM's significant −20.8% becomes null on both scales, so after adjustment no model is significantly cheaper under code mode. Sonnet becomes positive on the log scale and stays null on the additive one. Haiku and Qwen are essentially unchanged. The adjustment is itself an assumption. It requires one slot effect for both arms, and because every matrix shares the permutation it cannot separate a model-specific slot effect from a model effect. An arm-specific version is estimable but useless here, with standard errors of 0.145–0.710 log units.

Table 6: PTC against native on batch 1, raw (the first reading) and adjusted for execution order on the log and the additive scale, percent of native mean with 95% intervals.

	raw		adjusted, log		adjusted, additive	
model	est.	95% CI	est.	95% CI	est.	95% CI
GLM-5.3-Flash	−20.75	[−44.30, −2.79]	−8.92	[−19.97, +1.80]	−17.27	[−42.55, +0.41]
DeepSeek V4.1 Flash	+0.13	[−8.90, +9.76]	−0.78	[−10.02, +9.63]	+0.34	[−11.37, +11.59]
gpt-6-luna	+1.20	[−8.42, +11.29]	+6.65	[−6.47, +23.03]	+2.35	[−9.90, +15.98]
Claude Sonnet 5.5	+9.51	[−0.08, +18.01]	+17.02	[+5.05, +28.41]	+6.85	[−5.07, +16.82]
Qwen3.8-Flash	+30.67	[+10.02, +53.79]	+35.35	[+15.61, +58.32]	+32.39	[+12.94, +54.79]
Claude Haiku 4.5	+53.84	[+31.25, +76.73]	+66.13	[+39.06, +99.13]	+51.79	[+28.44, +75.63]

The remedy is cheap if applied before collection: randomise the order independently for every matrix and configuration, record the slot so it can be modelled, and for a two-arm contrast counterbalance which arm leads. Applied afterwards it costs a second collection.

A counterbalanced re-run. To test whether the shared order had biased the contrast, rather than only adjusting for it, we re-collected the central pair (`dsh:native` against `dsh:ptc`) on the four cheaper models on both suites and on Claude Haiku 4.5 on batch 1, 1,260 cells in all. In every matrix each arm led in exactly half of the (task, seed) blocks, under an assignment shuffled per matrix. The plan, its five questions and the decision rule below were filed before any re-run record existed. Its design and analysis were not amended; three tooling defects found during collection were fixed and recorded (Appendix D). The prediction was that, if the shared order had biased the raw contrasts, the re-run would lie closer to the order-adjusted estimate than to the raw one in at least 5 of the 8 pre-registered (model, suite) rows (at most 3 would refute it). It does in exactly 5 (Table 7), so the prediction holds, narrowly. In two of the five rows the margin is under 1.3 points, while in the row the plan singled out it is decisive: GLM's significant batch-1 saving of −20.8%, null after adjustment, did not return. The re-run puts GLM's batch-1 contrast at +15.4% [+1.0, +33.0]. Of the nine classifications, seven agree with the original raw reading and seven with the adjusted one. The order effect itself, measured as the contrast in PTC-first blocks minus the contrast in native-first blocks, has an interval that includes zero for eight of the nine rows. For Haiku it is −49.0% of native cost [−97.1, −3.2], which counterbalancing cancels from the contrast.

Table 7: The counterbalanced re-run of PTC against native, percent of native mean with 95% intervals: the filed raw and order-adjusted (log) estimates, the re-run's raw estimate, the row's verdict in the pre-registered test (which estimate the re-run lies closer to; Haiku was not one of the 8 rows), and the order effect (PTC-first minus native-first blocks).

suite	model	filed raw	filed adj.	re-run [95% CI]	closer to	order effect [95% CI]
batch 1	DeepSeek V4.1 Flash	+0.1	−0.8	+0.9 [−10.0, +12.7]	raw	+6.2 [−9.2, +22.2]
	GLM-5.3-Flash	−20.8	−8.9	+15.4 [+1.0, +33.0]	adjusted	−10.8 [−49.4, +22.4]
	Qwen3.8-Flash	+30.7	+35.3	+33.1 [+23.3, +42.5]	adjusted	−4.8 [−32.7, +20.4]
	gpt-6-luna	+1.2	+6.7	−2.1 [−13.6, +8.6]	raw	−2.5 [−25.7, +19.9]
	Claude Haiku 4.5	+53.8	+66.1	+78.4 [+57.0, +102.3]	(adjusted)	−49.0 [−97.1, −3.2]
tasks-b	DeepSeek V4.1 Flash	−21.8	−17.0	−16.8 [−29.8, −6.3]	adjusted	−15.8 [−38.9, +2.9]
	GLM-5.3-Flash	−10.1	−6.2	−3.8 [−16.5, +9.3]	adjusted	+1.0 [−16.1, +18.4]
	Qwen3.8-Flash	+5.7	+7.0	+12.9 [+0.6, +28.2]	adjusted	+18.0 [−9.6, +53.0]
	gpt-6-luna	−16.6	−16.2	−19.0 [−29.0, −9.4]	raw	−0.4 [−20.0, +19.0]

Two classifications change. GLM on batch 1 moves from negative to positive, and Qwen on tasks-b from null to positive. Both new intervals clear zero by about one point and both point estimates lie near the noise floor of Pitfall 6, so we read them as *not negative* and *not null* respectively, not as new effects. Native-arm cost per cell itself drifted by −8.1% to +8.0% between the collections. GLM's reversal from −20.8% to +15.4% is at least as consistent with that drift as with a shared slot effect. All other analyses in this paper use the original data with the shared order.

7 Pitfall in the outcome measure

7.1 Pitfall 5: cost without quality misleads

Pitfall 5. A cost difference between two configurations is a cost difference at equal quality only if they solve the same tasks, and this has to be tested, not assumed.

The pre-registered solve-rate tests covered the terseness contrasts, not code mode against native, so the central cost contrast carried no quality check. Table 8 adds it. On five models the two modes solve nearly the same cells. On Claude Haiku 4.5 they do not. Native solved 60 of 62 cells and PTC 46, with 16 cells solved by native only and 2 by PTC only (exact McNemar $p = 0.0013$). The first reading's +53.8% per task becomes +100.7% per solved task. On the 44 cells both modes solved, PTC still costs +67.8% [+36.7, +99.7]. So the cost penalty is not an artefact of failed runs, and the per-task figure hides a reliability loss altogether. On tasks-b the direction is the same (Haiku +6.2% per task, +62.4% per solved; Appendix C), but the per-task interval [−7.35, +19.67] and the both-solved interval (+8.52% [−11.27, +27.22]) both include zero. The counterbalanced re-run of Pitfall 4 reproduces it on batch 1 and sharpens it. Native solved 62 of 62 and PTC 48, with 14 cells solved by native only and none by PTC only ($p = 0.00012$). PTC costs +78.4% per task, +130.5% per solved task, and +107.1% [+72.7, +146.2] on the 48 cells both modes solved. Of the re-run's other eight rows, one shows a solve gap: GLM on tasks-b, 78 against 71 ($p = 0.039$), which a Holm correction over the nine rows would not keep.

Table 8: Quality check on the central contrast: cells solved by native/PTC (of 62 per arm on batch 1 and 80 on tasks-b; 36 for Haiku on tasks-b), discordant (task, seed) pairs (native only : PTC only) with the exact McNemar test, and the raw contrast on cells both modes solved (95% task-clustered bootstrap, $B = 200{,}000$).

suite	model	solved	discordant	McNemar p	cells	both-solved contrast
batch 1	DeepSeek V4.1 Flash	62/62	0 : 0	1.00	62	+0.13% [−8.90, +9.76]
	GLM-5.3-Flash	61/62	0 : 1	1.00	61	−16.03% [−30.47, −2.20]
	Qwen3.8-Flash	62/61	1 : 0	1.00	61	+32.32% [+12.55, +54.99]
	gpt-6-luna	60/61	1 : 2	1.00	59	−3.68% [−13.32, +5.54]
	Claude Sonnet 5.5	62/62	0 : 0	1.00	62	+9.51% [−0.08, +18.01]
	Claude Haiku 4.5	60/46	16 : 2	0.0013	44	+67.77% [+36.67, +99.66]
tasks-b	DeepSeek V4.1 Flash	74/77	0 : 3	0.25	74	−22.64% [−39.77, −9.19]
	GLM-5.3-Flash	72/74	5 : 7	0.77	67	−13.87% [−32.60, +0.82]
	Qwen3.8-Flash	75/78	1 : 4	0.38	74	+7.41% [−1.36, +16.29]
	gpt-6-luna	74/77	1 : 4	0.38	73	−18.01% [−25.14, −10.61]
	Claude Sonnet 5.5	78/77	1 : 0	1.00	77	−0.54% [−8.20, +6.18]
	Claude Haiku 4.5	26/17	10 : 1	0.012	16	+8.52% [−11.27, +27.22]

Graded on edited tests. Quality itself can be mismeasured. Batch 1 grades in the agent's workdir, and agents edit the tests they are graded by. On batch 1, gpt-6-luna did so in 81 of 619 analysed runs (13.1%), Haiku in 5, GLM in 2, Qwen in 1 and Sonnet in none. DeepSeek's records lack the field, so its rate is unmeasured rather than zero. Removing luna's edited runs moves its solve rate from 95.2% to 94.8% (510/538), and the worst case, in which every edited solve is treated as a fail, is 82.4% (510/619). On tasks-b, where grading uses a hidden suite and editing the visible tests cannot help, luna still edited them in 56.9% of runs. Grading against a suite the agent cannot touch removes this problem by construction.

8 Pitfall in inference

8.1 Pitfall 6: one suite, one sample, one resolution

Pitfall 6. A cost difference from one suite of about thirty tasks, without a measured noise floor or a replication, can be a property of that suite and that run.

The noise floor. We pre-declared an A/A design: one arm (`dsh:native` on DeepSeek) collected twice over all 31 tasks and both seeds, with byte-identical requests (verified for all 62 pairs). The two collections differ by −5.75% of per-cell cost, 95% CI [−15.6%, +3.4%]: an interval half-width of 9.5% (sd of per-task differences 26.8%). That half-width is the resolution of a two-arm cost contrast at this sample size. We suspected that part of the −5.75% was carry-over rather than noise: the passes ran back to back on a provider that caches automatically, and the second drew more input from cache (95.7% against 93.3%, with 34% fewer uncached tokens).

The re-run of Pitfall 4 included the interleaved A/A that separates the two: the same arm and an identical copy, run block by block inside one matrix, with which copy leads counterbalanced, again byte-identical in all 62 pairs. We predicted, in the filed plan, that the copies would differ by less than ±2% and that their cache-hit shares would differ by less than one point. The second part holds (95.5% against 95.2%), and carry-over, now measured directly as the second copy minus the first, is +1.7% [−7.9, +13.7]. The first part fails. The copies differ by −5.3% [−17.7, +5.6], an interval half-width of 11.6%. The earlier floor was not inflated by carry-over; it was, if anything, narrow. The variation is run to run, and the cache accounting does not explain it. Scaling the interval by $1/\sqrt{n}$, resolving ±5% would take about 170 paired tasks and ±3% about 470. Three of the six adjusted batch-1 effects lie inside either band (Figure 1).

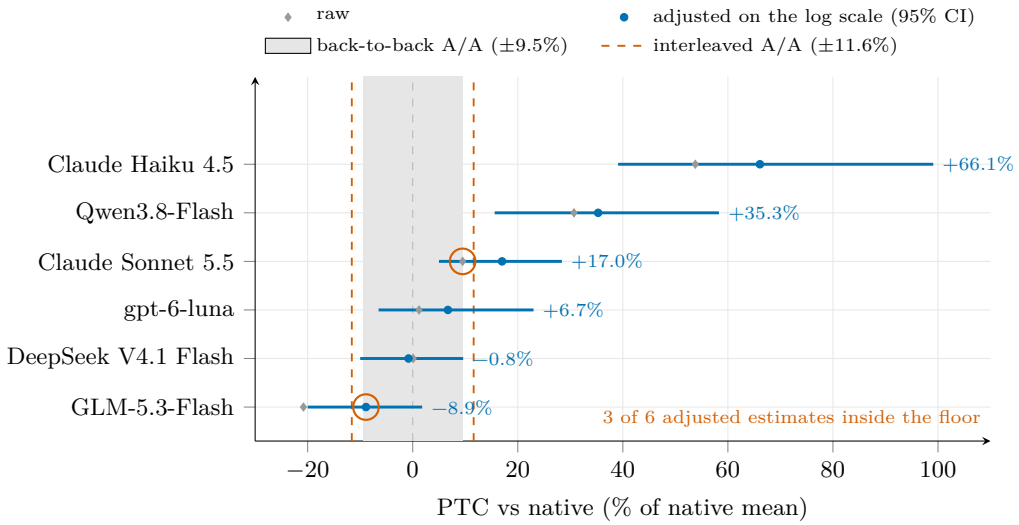


Figure 1: The batch-1 effects adjusted on the log scale (percent of native mean, 95% intervals; $n = 31$ tasks per model) against the back-to-back A/A half-width of 9.5% (shaded) and the interleaved A/A half-width of 11.6% (dashed); grey diamonds are the raw estimates. Negative values mean PTC is cheaper than native. GLM's adjusted −8.9% lies inside both, and Sonnet's raw +9.5% on the edge of the first and inside the second (ringed). Both floors are measured on DeepSeek only.

Resolution of the interval itself. The study's first analyses used $B = 5{,}000$ bootstrap resamples. At that resolution Sonnet's lower bound sat at +0.28%, a significant result, but it fell below zero in 25 of 40 reseeds. At $B = 200{,}000$ it is −0.08%. A classification that depends on the random seed of the bootstrap is not a finding.

The replication. After seeing the batch-1 results we filed a replication criterion on a second suite before any of its data existed. **R1**: each of four providers keeps its batch-1 raw classification (Qwen positive, GLM negative, luna null, DeepSeek null). **R2**: at least one provider is positive and one negative. Replication requires R1 for all four plus R2. Neither holds (Table 9). R1 is 0 of 4 and stays 0 of 4 across 20 bootstrap seeds. R2 fails because no provider is positive. Luna and DeepSeek moved from null to significantly negative, Qwen lost its positive result, and GLM's interval crossed zero by 0.14 points. The quality check leaves every tasks-b classification unchanged. On the log-adjusted scale the six-model spread fell from 75.1 points to 28.8; among the four pre-registered providers plotted in Figure 2 it is 24.0. Adjusting both suites for execution order does not repair the failure: R1 becomes 1 of 4 and R2 still fails. No single divergence survives every analysis convention, since on the additive scale all four pairs of intervals overlap. The pattern, however, does not reproduce under any of them. Nor is the suite dependence an artefact of the shared order: in the counterbalanced re-run (Table 7), as in the original, DeepSeek and luna are null on batch 1 and significantly negative on tasks-b.

Table 9: Pre-registered replication criterion R1: filed batch-1 classification against the tasks-b classification (raw contrast, 95% interval).

provider	batch 1 (filed)	tasks-b	tasks-b 95% CI	R1
Qwen3.8-Flash	positive	null	[−3.93, +15.01]	differs
GLM-5.3-Flash	negative	null	[−21.12, +0.14]	differs
gpt-6-luna	null	negative	[−24.35, −8.55]	differs
DeepSeek V4.1 Flash	null	negative	[−37.46, −9.38]	differs

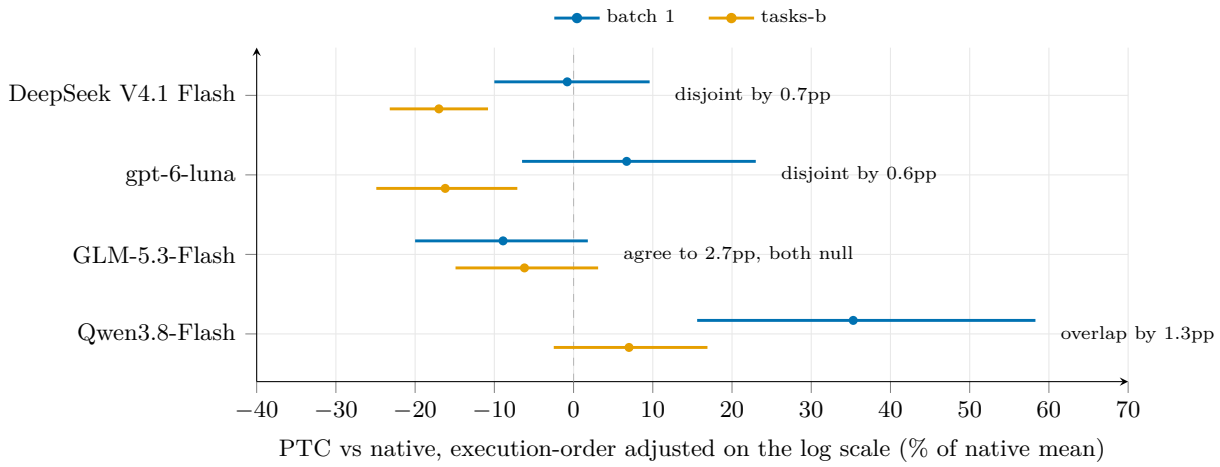


Figure 2: The replication: tasks-b against batch 1 for the four pre-registered providers, both adjusted for execution order on the log scale, with 95% intervals (batch 1: $n = 31$ tasks; tasks-b: $n = 40$). Negative values mean PTC is cheaper than native. On tasks-b the four providers span 24.0 points (Qwen +6.96 to DeepSeek =17.04).

Both Anthropic models, collected on tasks-b as an extension, also move from positive to null (Haiku +6.2% over 18 tasks, Sonnet −0.5% over 40). Three of the six pitfalls feed into this failure: the first reading's significant GLM result was partly an execution-order effect (Pitfall 4), Haiku's penalty is entangled with its failures (Pitfall 5), and most effects sit near the noise floor. Without the second suite, the first reading would have stood.

9 What survives: code mode after the corrections

With the six pitfalls addressed as far as the data allow, the following can be said about code mode.

The cost effect depends on the model and the suite. After adjusting for execution order, code mode's effect on batch 1 ranges from −8.9% to +66.1% (Appendix Figure 4), and the pattern did not replicate.

The counterbalanced re-run agrees: on batch 1 no model is significantly cheaper under code mode, while on tasks-b DeepSeek (−16.8%) and luna (−19.0%) are, as in the original collection. Those two savings are the only ones the study reproduces, and only on one suite. The one robust case is Claude Haiku 4.5 on batch 1, on which code mode is dearer per solved task and less reliable, in both collections (+78.4% per task and 48 against 62 solved in the re-run). On tasks-b the per-solved direction is the same (+62.4%) but the per-task and both-solved intervals include zero.

The price ratio does not explain it. The study was motivated by the idea that code mode shifts cost toward output, so its penalty should grow with the output:input price ratio. The pre-registered primary test (H1) is inconclusive: the slope of the penalty on output share is $b_1 = +0.00467$ [−0.00566, +0.01579], and on tasks-b it is negative. The pre-registered secondary, a gap of more than 5 points between the mean penalty at ratio 5.00 and at 3.33, is nominally met (42.3 points). But ratio 3.33 is one model, and the three models sharing ratio 5.00 differ among themselves by 59.5 of the 75.1-point adjusted spread (79.2%; Appendix Figure 4). The ratio is read off the provider cells, not manipulated, and is confounded with model and cache regime. These data neither establish nor exclude a ratio effect.

Code mode reduces requests, but that is not a cost saving. Native mode made more model requests per run than PTC on both suites, +2.228 [+1.228, +3.360] on batch 1 and +8.378 [+7.244, +9.606] on tasks-b, as pre-registered (H3). The batch-1 effect rests largely on gpt-6-luna (20.4 against 12.1 requests per run). Without it the interval includes zero, and Qwen and Haiku show no difference. Because native and PTC differ in whether calls can be batched as well as in tool count, we read H3 as "PTC reduces request count", not as an effect of schema count. Request savings did not predict cost savings: the two models with the largest savings on batch 1 have null cost effects.

Caching explains at most one model. Prefix retention, the share of a cached prefix the provider still serves from cache, falls under PTC only on Qwen (−0.065 [−0.078, −0.052]), and there it is confounded with breakpoint placement (Pitfall 2). On Haiku the retention loss is −0.005, far too small to explain its penalty. Elsewhere retention is flat at 0.99–1.00.

Harness rankings are only weakly shared across models. Over the eight non-terse arms, the median Spearman correlation between two models' cost rankings is 0.238 on batch 1. That is more than independent random rankings would give ($p = 0.047$), but on tasks-b the median is 0.024. Within a model, 74% of adjacent pairs in the cost ranking are statistically indistinguishable on batch 1 and 61% on tasks-b (54 pairs across all six models, including Anthropic external arms priced at the miss-rate default of Pitfall 5.3). Pairs of DSH arms tie in 18 of 24 on both suites, pairs of two external harnesses in half (Figure 3). Adjusting for execution order changes these figures by at most two points. A ranking measured with one model is a weak guide to another, and most steps between harnesses are not resolved at this sample size.

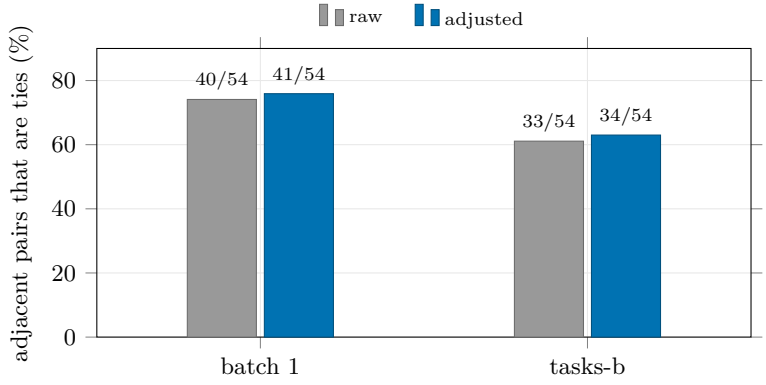


Figure 3: Adjacent arm pairs in each model's cost ranking that are statistically indistinguishable, raw and adjusted for execution order (ties out of 54 adjacent pairs). The 54 pairs include Anthropic external arms priced at the miss-rate default used when the cache split is missing.

10 A checklist for measuring agent cost

Table 10 collects one practice per pitfall. None is expensive if planned before collection. Several are expensive to repair afterwards, which is the situation this study was in.

Table 10: Practices for measuring the cost of agents, one per pitfall.

	pitfall	practice
1	Token volume is not cost	Price from the provider's own usage block, including the cache split; report billed and cold-cache cost side by side.
2	Cache configuration is part of the treatment	Report each condition's cache regime (automatic, declared, where the breakpoints sit); check that no arm falls under a provider's minimum cacheable length; never inject declarations into a client that declares its own.
3	Accounting can be missing; meters fail silently	Reconcile computed cost against the provider's ledger; make cost tools fail on unknown models or missing cache splits instead of defaulting; report brackets where accounting is missing.
4	Execution order carries cost	Randomise the run order independently per configuration and matrix; counterbalance which arm leads in a paired contrast; record the slot; check for order effects before analysing.
5	Cost without quality misleads	Test whether arms solve the same tasks; report cost per solved task and the contrast on commonly solved tasks; grade against suites the agent cannot edit.
6	One suite, one sample, one resolution	Run an interleaved A/A to measure the noise floor and size the task set from it; use enough bootstrap resamples to make classifications seed-stable; replicate on a second suite before claiming a direction.

11 Use of AI tools

The authors used large language model (LLM) assistants extensively in this work. They helped design and set up the experiments and guided their execution; they also helped build the surrounding infrastructure, including tooling and the measurement harness. The first author wrote an initial rough draft of the manuscript, which LLM assistants substantially rewrote and expanded into the present text. The authors directed the project and chose the research questions; Alexander Feurer contributed substantially to the initial research idea; Max Reischl ran and oversaw the experiments. Both authors reviewed the final manuscript and take full responsibility for all content, numbers and citations. LLM assistants are not authors of this paper.

12 Limitations

One harness carries the code-mode contrast. Code mode was tested inside DSH only, so the code-mode findings concern DSH's implementation, and we wrote its study conditions. The pitfalls do not depend on this, but the code-mode results would be stronger with a second harness that offers a code mode.

Model, provider and cache regime are confounded. Each model comes from one provider with one cache regime and wire protocol. Only within Anthropic can model be separated from provider, and there the adjusted effect still differs by 49 points.

The study carries its own pitfalls. The shared execution order affects both suites, and the adjustment for it rests on an assumption the design cannot test. The counterbalanced re-run removes this for the central

contrast on five models, but not for Sonnet, the eight other arms or the harness rankings, which still rest on the adjustment. The re-run is a later collection, so it also carries drift (native cost per cell moved by up to 8%). The sample is below what the measured noise floor calls for, and both floors are measured on one model. Haiku's tasks-b row covers 353 analysed cells from 18 of 40 tasks that are not a random subset. The pre-registration was amended 23 times, so its freeze is nominal (Appendix A). Several figures circulated during the study were later withdrawn (Appendix E). The pilots behind Pitfall 2 and the incidents in Pitfall 3 come from the development record and are not part of the checksummed artifact.

Lost data. An earlier, larger panel, including a seventh model, was destroyed on 2026-10-02 together with its logs. Nothing here depends on it.

13 Conclusion

A careful study of whether code mode makes coding agents cheaper—protocol fixed in advance with 23 documented amendments (Appendix A)—reached a mixed answer. No model was significantly cheaper on batch 1; on the second suite DeepSeek and gpt-6-luna were, and a counterbalanced re-run reproduced both savings. The batch-1 pattern did not replicate. Reaching even that answer required avoiding six pitfalls, and at each of them a standard measurement practice would have changed the conclusion or mis-sized it. Priced without cache accounting, code mode looks cheaper on 4 of 6 models (Table 3); the point-estimate sign flips on half, and those three billed intervals include zero (only Haiku survives Holm on batch 1). Under one cache declaration it is never cached. Without an outside ledger, the cost tool reports more than was spent. With a shared run order, a significant saving appears that neither the adjustment nor a counterbalanced re-run supports (the order effect itself is significant in 1 of 9 re-run rows; GLM's reversal sits within native-arm drift). Without a quality check, a doubling of cost per solved task looks like +54%. Without a second suite, the first suite's pattern would have stood. Cost is now routinely reported for agents. It should be measured with the same care as accuracy.

References

Anthropic. Introducing advanced tool use for Claude. Anthropic research page, 2025a. URL https://www.anthropic.com/research/advanced-tool-use. Vendor page; fetched 2026-11-06; formerly under /engineering/.

Anthropic. Code execution with MCP: Building more efficient agents. Anthropic Engineering blog, 2025b. URL https://www.anthropic.com/engineering/code-execution-with-mcp. Vendor engineering blog, published 2025-11-04.

Anthropic. OpenAI SDK compatibility. Claude Platform documentation, 2025c. URL https://platform.claude.com/docs/en/cli-sdks-libraries/libraries/openai-sdk. Vendor documentation; fetched 2026-10-10.

Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2604.22750. arXiv:2604.22750.

Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. On Randomness in Agentic Evals. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2602.07150. arXiv:2602.07150.

Robert E. Blackwell, Jon Barry, and Anthony G. Cohn. Towards Reproducible LLM Evaluation: Quantifying Uncertainty in LLM Benchmark Scores. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2410.03492. arXiv:2410.03492.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=cSimKw5p6R. TMLR 2024; arXiv:2305.05176.

Lingjiao Chen, Chi Zhang, Yeye He, Ion Stoica, Matei Zaharia, and James Zou. The Price Reversal Phenomenon: When Cheaper Reasoning Models Cost More. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2603.23971. arXiv:2603.23971.

cline. LiteLLM / OpenAI-Compatible providers never emit cache_control — no client-side way to enable Anthropic prompt caching (Issue #13667). GitHub issue, cline/cline, 2026. URL https://github.com/cline/cline/issues/13667. Issue tracker report; fetched 2026-10-10.

Cloudflare. Code Mode: give agents an entire API in 1,000 tokens. Cloudflare blog, 2026. URL https://blog.cloudflare.com/code-mode-mcp/. Vendor blog post; fetched 2026-10-10.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. In *International Conference on Learning Representations 2024 (ICLR 2024)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/b47d93c99fa22ac0b377578af0a1f63a-Abstract-Conference.html. ICLR 2024; arXiv:2404.14618.

Mehmet Hamza Erol, Batu El, Mirac Suzgun, Mert Yuksekgonul, and James Zou. Cost-of-Pass: An Economic Framework for Evaluating Language Models. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2504.13359. arXiv:2504.13359.

Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An Empirical Study of Harness Design for Coding Agents. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2609.20804. arXiv:2609.20804.

In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems 6 (MLSys 2024)*, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html. MLSys 2024; arXiv:2311.04934.

Jiwoo Han and Yuekai Sun. How good is your harness? Statistical evaluation of coding harnesses. In *Workshop on Combining Theory and Benchmarks (CTB), ICML 2026*, 2026. URL https://icml.cc/virtual/2026/68323. ICML 2026 CTB workshop; also OpenReview forum id QI8z3skBwt.

Nicolas Hrubec and Jürgen Cito. Reducing Token Usage of State-in-Context Agents using Minification. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2606.01326. arXiv:2606.01326.

Yiran Hu, Nan Jiang, Shanchao Liang, Anik Dey, Yi Wu, and Lin Tan. Analyzing and Mitigating Cost-Inefficient Behaviors in Coding Agents. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2609.30725. arXiv:2609.30725; under review.

Thibault Jaigu. The Coding Agent Economy: The Economics of Coding Agents — Cost, Usage, and Model Preferences Across AI-Native IDEs. Requesty industry report, May 2026, 2026. URL https://www.requesty.ai/coding-agent-economy. Industry report.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1658–1677. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.91. URL https://aclanthology.org/2024.acl-long.91/. ACL 2024; DOI: 10.18653/v1/2024.acl-long.91.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations 2024 (ICLR 2024)*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66. ICLR 2024; arXiv:2310.06770.

Berk Kalelioğlu and Cem Dilmegani. A-CODE-CLI Bench: Agentic CLI Benchmark (Agentic CLI Tools Compared: Claude Code vs Cline vs Aider). AIMultiple benchmark, 2026. URL https://aimultiple.com/agentic-cli. Vendor benchmark page, updated 2026-06-29.

Alibek T. Kaliyev and Artem Maryanskyy. How Much Coordination Gain Is Real? A Paired Noise-Floor Protocol for Multi-Agent LLM Benchmarks. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2606.20695. arXiv:2606.20695.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI Agents That Matter. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2407.01502. arXiv:2407.01502.

Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2510.11977. arXiv:2510.11977.

Ted Kwartler, Alan Aqrawi, and Arian Abbasi. Harness Tokenomics: A Router for the Enterprise Agentic Control Plane. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2609.28919. arXiv:2609.28919.

Sydney Lewis. Same Model, Different Harness: Different Coding-Agent Results. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2608.26218. arXiv:2608.26218.

LiteLLM team. Auto Router: Opus level quality at up to 27% lower cost. LiteLLM documentation/blog, 2026. URL https://docs.litellm.ai/blog/auto-router-terminal-bench-benchmark. Vendor benchmark page; fetched 2026-10-10.

Fengshuo Liu, Ying Liu, Ruize Sun, Lie Luo, and Siyuan Guo. Coding Agents Have Converged: Why the SWE-bench Leaderboard Can No Longer Order Its Top Entries, and What to Measure Instead. In *ADMA 2026 (International Conference on Advanced Data Mining and Applications), Special Session on Responsible Data Intelligence*, 2026. URL https://arxiv.org/abs/2609.17394. ADMA 2026; arXiv:2609.17394.

Yangze Liu and Zhongyi Han. What Does a Harness Buy? Tokens, Mostly. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2610.04433. arXiv:2610.04433.

Elias Lumer, Vamse Kumar Subbiah, James A. Burke, Pradeep Honaganahalli Basavaraju, and Austin Huber. Toolshed: Scale Tool-Equipped Agents with Advanced RAG-Tool Fusion and Tool Knowledge Bases. In *Proceedings of the 17th International Conference on Agents and Artificial Intelligence (ICAART 2025), Volume 3*, pp. 1180–1191. SciTePress, 2025. doi: 10.5220/0013303000003890. URL https://doi.org/10.5220/0013303000003890. DOI: 10.5220/0013303000003890; preprint arXiv:2410.14594.

Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. Don't Break the Cache: An Evaluation of Prompt Caching for Long-Horizon Agentic Tasks. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2601.06007. arXiv:2601.06007.

Hazel Mak, Susheel Suresh, Sahil Bhatnagar, Barry Wang, Chhaya Methani, and Alejandro Gutierrez Munoz. Is Bash All You Need? An Empirical Study of Tool Interfaces for Enterprise Digital Worker Agents. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2609.11999. arXiv:2609.11999.

Abasiono Mbat, Oselumese Agbonrofo, Oladimeji Abaniwonnda, and Samuel Oyefusi. Benchmarking LLM Tool Orchestration in Resource-Constrained Agentic Workflows: Native MCP Function Calling vs Code Mode. *International Journal of Computer Applications*, 187(136), August 2026. URL https://ijcaonline.net/archives/volume187/issue136/benchmarking-llm-tool-orchestration-in-resource-constrained-agentic-workflows-native-mcp-function-calling-vs-code-mode/. IJCA 187(136), August 2026; PDF verified 2026-11-06.

Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models. In *International Conference on Learning Representations 2025 (ICLR 2025)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/ec2e7a896f8250986b3907f57621ce94-Abstract-Conference.html. ICLR 2025; arXiv:2410.05229.

Vinoth Nageshwaran, Soundararajan Ezekiel, Tin T. Tran, and V. Lakshmi Narasimhan. Large language model agent evaluation and benchmarking: a systematic survey, meta-taxonomy, and critical research roadmap. *Artificial Intelligence Review*, 2026. doi: 10.1007/s10462-026-11678-4. URL https://doi.org/10.1007/s10462-026-11678-4. DOI: 10.1007/s10462-026-11678-4.

Franck Ndzomga. Efficient Benchmarking of AI Agents. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2603.23749. arXiv:2603.23749.

William Nixon, Muxin Tian, Yunjia Zheng, Haryadi S. Gunawi, and Juncheng Yang. FreeInference Agentic Trace. Hugging Face dataset `harvardMadsys/freeinference_agentic_trace`, 2026. URL https://huggingface.co/datasets/harvardMadsys/freeinference_agentic_trace. Dataset accompanying "From Requests to Sessions: A Large-Scale Characterization of Human-Driven Agentic Workloads"; CC BY 4.0.

Eduardo Ari no de la Rubia and Szilard Pafka. Identical Runs, Different Results: Benchmarking AI Coding Agents on Open-Weight Models. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2609.33812. arXiv:2609.33812.

NousResearch. [Bug]: Prompt caching never engages for LiteLLM on the OpenAI-compatible wire (Issue #84506). GitHub issue, NousResearch/hermes-agent, 2026. URL https://github.com/NousResearch/hermes-agent/issues/84506. Issue tracker report; fetched 2026-10-10.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to Route LLMs from Preference Data. In *International Conference on Learning Representations 2025 (ICLR 2025)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html. ICLR 2025; arXiv:2406.18665.

OpenRouter. Prompt Caching. OpenRouter documentation, 2026. URL https://openrouter.ai/docs/guides/best-practices/prompt-caching. Vendor documentation; fetched 2026-11-06.

Ishan Patel, Sahil Sen, Elias Lumer, and Vamse Kumar Subbiah. The Bitter Lesson of Tool Calling. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2608.06370. arXiv:2608.06370.

Nearchos Potamitis, Vansh Ramani, Har Ashish Arora, Dhairya Kuchhal, Lars Klein, and Akhil Arora. ReasonBENCH: Benchmarking the (In)Stability of LLM Reasoning. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2512.07795. arXiv:2512.07795.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations 2024 (ICLR 2024)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html. ICLR 2024; arXiv:2307.16789.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are Emergent Abilities of Large Language Models a Mirage? In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. doi: 10.52202/075280-2425. URL https://papers.neurips.cc/paper_files/paper/2023/hash/adc98a266f45005c403b8311ca7e8bd7-Abstract-Conference.html. NeurIPS 2023; DOI: 10.52202/075280-2425; arXiv:2304.15004.

Strands Agents team. strands-code-agent: CodeAct as the action paradigm for Strands Agents. Strands Agents (AWS) documentation; announcement post dated 2026-07-15, 2026. URL https://strandsagents.com/docs/integrations/agent-extensions/strands-code-agent/. Vendor documentation.

SWE-agent team. mini-SWE-agent: The 100 line AI agent that's actually useful. Project documentation, 2025. URL https://mini-swe-agent.com/latest/. Documentation site; fetched 2026-10-10.

Naman Vats and Oleg Golev. The Scaffold Effect in Coding Agents: Harness Choice as a Hidden Variable in Coding-Agent Evaluation. In *5th Deep Learning for Code Workshop (DL4C) at ICML 2026*, 2026. URL https://arxiv.org/abs/2607.22585. arXiv:2607.22585; workshop paper (non-archival).

Ningning Wang, Xavier Hu, Pai Liu, He Zhu, Yue Hou, Heyuan Huang, Shengyu Zhang, Jian Yang, Jiaheng Liu, Ge Zhang, Changwang Zhang, Jun Wang, Yuchen Eleanor Jiang, and Wangchunshu Zhou. Efficient Agents: Building Effective Agents While Reducing Cost. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2508.02694. arXiv:2508.02694.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, 2024. URL https://arxiv.org/abs/2402.01030. ICML 2024; arXiv:2402.01030.

Sarel Weinberger and Amir Hozez. Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2608.01347. arXiv:2608.01347.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754. URL https://doi.org/10.1145/3715754. FSE 2025 / PACMSE; DOI: 10.1145/3715754; also known as Agentless; arXiv:2407.01489.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2412.14161. arXiv:2412.14161.

Xiaonan Xu and Wenjing Wu. Validation Evidence in LLM Repair Agents: How Much of What Passes Actually Tests the Bug? *arXiv preprint*, 2026. URL https://arxiv.org/abs/2607.28871. arXiv:2607.28871.

Hong Yang, Qi Yu, and Travis Desell. When Does Restricting a Coding Agent to execute_code Help? A Regime × Agent-Design Ablation. In *Agentic Software Engineering (SE 3.0) Workshop at KDD 2026 (non-archival)*, 2026a. URL https://arxiv.org/abs/2607.10569. arXiv:2607.10569.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL https://neurips.cc/virtual/2024/poster/93753. NeurIPS 2024 poster; arXiv:2405.15793.

Pei Yang, Wanyi Chen, Tongyun Yang, Pengbin Feng, Jiarong Xing, Wentao Guo, Yuhang Yao, Yuhang Han, Hanchen Li, Xu Wang, Yuan Gao, Zeyu Wang, Jie Xiao, Anjie Yang, Liang Tian, Lynn Ai, Eric Yang, and Tianyu Shi. TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing. *arXiv preprint*, 2026b. URL https://arxiv.org/abs/2605.18859. arXiv:2605.18859.

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, pp. 94–109. ACM, 2025. doi: 10.1145/3689031.3696098. URL https://doi.org/10.1145/3689031.3696098. EuroSys 2025; DOI: 10.1145/3689031.3696098; arXiv:2405.16444.

Ying Zhang, Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li, Yepang Liu, and Leo Zhang. Silent and Costly: Characterizing and Detecting Prompt-Caching Mis-Integration in LLM Agents. *arXiv preprint*, 2026a. URL https://arxiv.org/abs/2610.34817. arXiv:2610.34817.

Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop Comparing LLM Agents Without Disclosing the Harness. *arXiv preprint*, 2026b. URL https://arxiv.org/abs/2605.23950. arXiv:2605.23950.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*, pp. 1592–1604. ACM, 2024. doi: 10.1145/3650212.3680384. URL https://doi.org/10.1145/3650212.3680384. DOI: 10.1145/3650212.3680384; arXiv:2404.05427.

Lu Zhao, Christopher M. Poskitt, and David Lo. Pairing Matters: Performance, Failures, and Efficiency in Lightweight Coding Agents. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2610.32131. arXiv:2610.32131.

Mengyu Zheng, Kai Han, Boxun Li, Haiyang Xu, Yuchuan Tian, Wei He, Hang Zhou, Jianyuan Guo, Hailin Hu, Lin Ma, Chao Xu, Guohao Dai, Lixue Xia, Yunchao Wei, Yunhe Wang, and Yu Wang. Claw-SWE-Bench: A Benchmark for Evaluating OpenClaw-style Agent Harnesses on Coding Tasks. *arXiv preprint*, 2026a. URL https://arxiv.org/abs/2606.12344v1. arXiv:2606.12344v1 (version 1 is the one cited).

Weihuang Zheng, Tianyuan Zou, Eileen Ye, Alphet Liu, Youyong Kong, Ya-Qin Zhang, Duran Zheng, and Maxm Pan. E-Bench: Benchmarking Multi-Step Tool-Use Agents in Real-World Product Scenarios. *arXiv preprint*, 2026b. URL https://arxiv.org/abs/2607.23722. arXiv:2607.23722.

Appendix A Pre-registration status

Table 11: Confirmatory hypotheses on batch 1 (six models) and their verdicts under the filed decision rules.

hypothesis	result	verdict
H1 (primary): the PTC penalty rises with output share	$b_1 = +0.00467$ [−0.00566, +0.01579], $n = 186$	inconclusive
H2: terseness reduces the PTC penalty	mean $d = +0.00151$ [−0.00010, +0.00354], Wilcoxon $p = 0.80$	inconclusive
H3: 24 schemas rather than 1 increase requests	+2.228 [+1.228, +3.360]	supported
H4: cache hit rate does not predict arm cost	$b_C = +0.11157$ [+0.06235, +0.17140]	refuted

- H5 (exploratory, no formal test) predicted that harness cost rankings are not stable across providers; §9 reports the statistics.
- The filing was amended 23 times. Versions 1–3 each named a freeze commit that did not contain them, and version 4 edited the text after its tag. Amendments 21–23 post-date the last tag.
- H1 was refuted on the four cheaper providers and became inconclusive when the Anthropic models were added. A mixed model with a task random intercept gives the same b_1. On the four pre-registered providers on tasks-b, $b_1 = =0.0125$ [−0.0203, −0.0050].
- The Holm family was filed with 12 members (4 cost, 8 solve-rate) and grew to 18 when the Anthropic models were added, after their collection. Of the 18, two survive, both Haiku's: the PTC cost contrast ($p = 0.00005$) and the PTC-to-PTC-terse solve-rate contrast ($p = 0.00014$). Qwen's cost contrast ($p = 0.0056$) fails only through the correction. On tasks-b the filed family of 12 has two survivors, DeepSeek ($p = 0.0001$) and luna ($p = 0.0003$), both saying PTC is cheaper.
- The filing exempts only H1 and H2 from correction by name. H3 is reported uncorrected, and its p of about 3×10^{-5} survives any family we could construct.
- The test-editing rule (report as measured, exclude nothing, publish two sensitivities) was committed before its effect was computed. The replication criterion and the Anthropic extension verdict were filed before any tasks-b record existed.
- The counterbalanced re-run (§6.1, Appendix D) was filed after the original results were known but before any re-run record existed, and was not amended. Of its two predictions, the order-bias prediction holds at the threshold (5 of 8) and the A/A prediction fails in its first part (§8.1).

Appendix B Robustness

Scale of the execution-order adjustment. By held-out prediction, the log scale wins 9 of 12 model×suite comparisons, by up to 1.8×. Four of those wins, and all three additive wins, are marginal. The scales disagree about a classification in two rows: GLM (null on both adjusted scales) and Sonnet (positive on log, null on additive).

Arm string. With `dsh:ptc-opt` against `dsh:native-opt` as the central pair, every tasks-b sign is kept (DeepSeek −19.4%, GLM −7.0%, Qwen +3.9%, Haiku +10.2%, Sonnet +1.4%, luna −8.8%), but luna moves from negative to null. On batch 1 the pairs differ in size and sometimes in sign (Haiku +53.8% against +79.0%, GLM −20.8% against −12.7%, DeepSeek +0.1% against −10.1%). Under the variant pair no provider's batch-1 and tasks-b intervals separate.

Seeds. On batch 1, Qwen is positive on seed 1 (+37.4% [+18.7, +57.3]) and null on seed 2 (+24.0% [−3.5, +58.3]). Haiku is positive on both, and Sonnet only on seed 2. On tasks-b, GLM is negative on seed 1 and null on seed 2.

Leave one model out. The adjacent-step tie fraction stays within 71.1–80.0% (batch 1) and 57.8–64.4% (tasks-b). The median ρ ranges over 0.119–0.262 and −0.190 to +0.071. H3 on batch 1 ranges from 2.206 to 2.703 for five of the six drops, and is +1.010 [−0.081, +2.223] without luna. On tasks-b it ranges from 6.379 to 9.642.

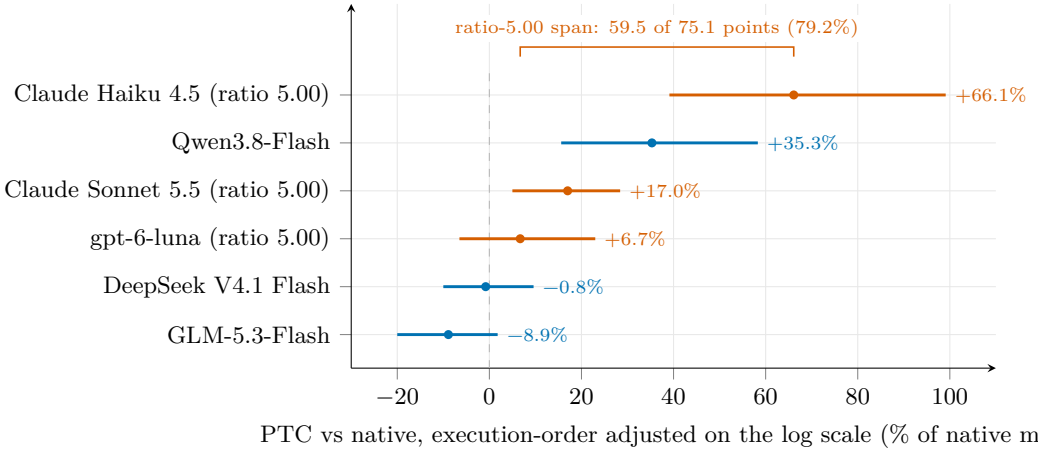


Figure 4: PTC against native on batch 1, adjusted for execution order on the log scale (percent of native mean, 95% intervals; $n = 31$ tasks). Negative values mean PTC is cheaper than native. The three models with an output:input price ratio of 5.00 (red) span 59.5 of the 75.1-point range (79.2%).

Replication under the adjustment. On the log-adjusted scale luna moved from +6.7% to −16.2% and DeepSeek from −0.8% to −17.0%, with intervals disjoint by 0.6 and 0.7 points. Qwen fell from +35.3% to +7.0%, and GLM moved from −8.9% to −6.2%, null on both suites.

Appendix C The Anthropic extension on tasks-b

Table 12: Anthropic extension on tasks-b, per cell and per solved task. Haiku's cells are those that ran before the account limit (353 analysed cells from 18 of 40 tasks).

arm	cells	solved	per cell	per solved task
Haiku `dsh:native`	36	26 (72.2%)	$0.2851	$0.3948
Haiku `dsh:ptc`	36	17 (47.2%)	$0.3028	$0.6412
Sonnet `dsh:native`	80	78 (97.5%)	$0.1363	$0.1398
Sonnet `dsh:ptc`	80	77 (96.3%)	$0.1356	$0.1409

The tasks-b plan required this per-solved-task reading for the Haiku row before the full matrix ran. Over the full 40-task design, counting cells lost to the outage as unsolved, 10 pairs are solved by native only and 1 by PTC only (exact McNemar $p = 0.0117$). Haiku's tasks-b interval is [−7.35, +19.67] and Sonnet's [−7.92, +5.98].

Appendix D The counterbalanced re-run

The re-run used the same harness version, tasks, grading, seeds and gateway as the original, with DSH only, so that no external harness was spent on. Its matrices: batch 1 (31 tasks, 2 seeds, 124 cells) on DeepSeek, GLM, Qwen, luna and Haiku; tasks-b (40 tasks, 2 seeds, 160 cells) on the four cheaper models; and the interleaved A/A on DeepSeek (124 cells). Every matrix was collected in full. The re-run cost $46.48 at the gateway meter against a filed stop of $70. For Haiku, the only matrix on its provider account in that window, the account balance fell by $20.61 ($57.90 to $37.29, read from the provider console) against $20.60 at the

meter: on the DSH arms, whose requests carry the cache split, meter and bill agree, unlike the external harnesses of §5.3.

Table 13: Quality in the re-run: cells solved by native/PTC (of 62 per arm on batch 1 and 80 on tasks-b), discordant pairs (native only : PTC only) with the exact McNemar test, and the contrast on cells both modes solved (95% task-clustered bootstrap, $B = 200{,}000$).

suite	model	solved	discordant	McNemar p	cells	both-solved contrast
batch 1	DeepSeek V4.1 Flash	62/62	0 : 0	1.00	62	+0.93% [−9.96, +12.67]
	GLM-5.3-Flash	62/62	0 : 0	1.00	62	+15.38% [+0.97, +32.95]
	Qwen3.8-Flash	62/60	2 : 0	0.50	60	+36.32% [+24.57, +49.00]
	gpt-6-luna	61/61	1 : 1	1.00	60	−1.31% [−11.29, +8.81]
	Claude Haiku 4.5	62/48	14 : 0	0.00012	48	+107.05% [+72.67, +146.15]
tasks-b	DeepSeek V4.1 Flash	78/78	2 : 2	1.00	76	−19.08% [−34.61, −6.65]
	GLM-5.3-Flash	78/71	8 : 1	0.039	70	−2.29% [−17.25, +14.80]
	Qwen3.8-Flash	74/74	3 : 3	1.00	71	+11.52% [+1.01, +22.70]
	gpt-6-luna	76/75	5 : 4	1.00	71	−23.59% [−32.82, −14.93]

Three tooling defects surfaced during collection, each stopped by the pipeline before it could affect a record, and each fixed and recorded in the filed plan. The first A/A smoke failed at its pilot because a process the harness spawned still held the shared A/A workdir when the twin started, so its clean is now retried until the handle is released. The pilot stage then judged records left in a shared scratch file by an earlier attempt, so each pilot is now judged on its own records only. And the collector priced Haiku at an all-harness Anthropic average, which put the matrix above its filed ceiling before it started, so the plan now carries Haiku's filed DSH cost. Separately, the Qwen tasks-b collection stopped at 124 of 160 cells when its processes were lost, and it was resumed under the same matrix identifier, so the counterbalanced assignment held. One cell that the interruption left with a malformed record was run again under the plan's retry rule. Its two records are averaged, as for every repeated cell.

Appendix E Corrections to earlier figures

Figures from earlier drafts of this study that are withdrawn or corrected:

- An execution-order "standardisation" that roughly halved every effect (Haiku +25.41%) used an estimator that summarised both arms' records as one arm's. It is withdrawn. Both legitimate corrections, a slot fixed-effect regression (+52.97%) and the paired estimator used here (+51.79% additive), leave Haiku near its raw value.
- An earlier A/A pair compared two differently named arms whose first requests differed by 20–30 bytes. Its floors (46.2%, then 7.7%) are withdrawn in favour of the repeated-baseline design of §8.1.
- External-harness prefix retention on Anthropic (0.560, 0.451) mixed four unmeasured external arms, coded as 0, with four DSH arms. No external-harness retention on the OpenAI-compatible path exists.
- Anthropic per-run costs of \$0.105–0.233 (Haiku) and \$0.082–0.202 (Sonnet) were unreconciled per-model bounds. Table 5 gives the ledger-reconciled brackets.
- Sonnet's batch-1 contrast was reported as significant at $B = 5{,}000$. It is null at $B = 200{,}000$.
- An earlier draft headlined a sign disagreement between providers and a penalty independent of the price ratio. The sign disagreement did not replicate and does not survive the execution-order adjustment, and the ratio question is open (§9).

Appendix F Artifact and reproducibility

Every number in the main matrices, the replication suite, the A/A run and the OpenCode pilot is re-derived from archived run records and gateway logs by commands recorded in `artifact/MANIFEST.json`, which checksums 126 files across two scopes: the batch-1 matrices as signed off on 2026-10-04, and the later material (the tasks-b suite, the execution-order adjustment, the ablations, the mixed-effects fit, the ranking and retention sensitivities, the test-edit incidence, the verdict seed stability, the A/A run, the retention pilot, the Anthropic cost bracket, the common-slot check, the quality check, the batch-1 contrasts at $B = 200{,}000$, and the counterbalanced re-run with its interleaved A/A). `node tools/artifact.mjs --verify` fails on any drift. The per-model Anthropic brackets are arithmetic on the filed bracket and the ledger (§5.3). The Anthropic cache A/B (Table 4), the billing reconciliation of §3.4 and the pipeline incidents of §5.3 come from the development pilots, whose records ship as documents but whose raw logs are not in the manifest.

Table 14: Where each result is filed (all under `analysis/`).

result	output (tool)
Tables 2, 9; family tests	`panel-analysis-6model-2026-10-09.json`, `panel-analysis-batchb-four-2026-10-09.json` (`panel-analysis.mjs`)
Table 6; Figures 4, 2	`position-adjust-6model-2026-10-09.json`, `position-adjust-batchb-2026-10-09.json` (`position-adjust.mjs`)
Table 8	`quality-guard-2026-10-10.json` (`quality-guard.mjs`)
Table 3, retention	`prefix-retention.json`, `prefix-retention-ci-2026-10-10.json`
Replication verdict, seed stability	`replication-verdict-batchb-2026-10-09.json`, `verdict-seed-stability-2026-10-10.json`
H1–H4, Table 11	`hypotheses-6model-2026-10-09.json`, `mixed-effects-6model-2026-10-04.json`
Rankings, Figure 3	`ranking-adjusted-2026-10-10.json`, `ablations-2026-10-09.json`
Table 5, OpenCode pilot	`anthropic-cost-bracket-2026-10-10.json`, `retention-pilot-2026-10-10.json`
A/A, Figure 1	`aa-placebo-2026-10-10.json` (`aa-analysis.mjs`)
Test editing	`test-edit-sensitivity-6model-2026-10-04.json`, `test-edit-incidence-2026-10-10.json`
Common-slot check	`common-f-infeasibility-2026-10-10.json`
Tables 7, 13; interleaved A/A	`rerun-compare-2026-10-11.json` (`rerun-compare.mjs`)

The figures are generated from these files by `node tools/figures.mjs`. The methods record, the pre-registration with its amendments, the replication plan and the defect records ship with the artifact. The long-form research record of the study, with every intermediate and withdrawn result, is `PAPER.md`.

Code and data availability. The gateway, run records, logs, analysis code and checksummed artifact are at https://github.com/Max6617/betterharness, commit `71a6627` (`71a662728c768782e55c91a1e94775f75f8a9956`), under the MIT License (Copyright (c) 2026 Max Reischl and Alexander Feurer). [Zenodo DOI to be added on release].